50.021: Artificial Intelligence Project Report

1009942 Yong Jun Han

22 April 2025

1 Introduction

This document is a report detailing my project developed under the theme "AI for Good", focusing on the use of Convolutional Neural Networks (CNNs) to classify images as either AI-generated or real.

The report will detail the problem at hand, dataset used, the CNN architecture, training methodology, evaluation metrics, and results, along with a GUI demonstration.

2 Problem Description

The rise of powerful generative AI models, such as GANs (Generative Adversarial Networks) and diffusion models, has led to the widespread creation of hyper-realistic synthetic images. While these technologies have enabled significant progress, they also pose a threat to information authenticity, with AI-generated images being used for misinformation, identity fraud, and other malicious purposes.

A recent case example of generative AI being used to deceive would be the recent incident of a French woman being duped out of hundreds of thousands of Euros, by an individual posing as American actor Brad Pitt. The scammer used AI-generated images and deepfake videos to convincingly impersonate the celebrity, building a deceptive online relationship and ultimately exploiting the victim emotionally and financially. This case highlights how AI-generated content can be misused for manipulation, especially when individuals are unable to verify the authenticity of what they see online.

This project will therefore address this need by developing an image classification model utilising a Convolutional Neural Network (CNN) to do binary clas-

sification of images as either real, or AI-generated, helping protect individuals and society from the misuse of synthetic imagery.

3 Dataset Processing

The dataset used in the project is the CIFake: Real and AI-Generated Synthetic Images from Kaggle.

Within the dataset are 120,000 colour images, split into 20,000 (10,000 per class) for testing and 100,000 (50,000 per class) for training.

With the images of the dataset being uniform, preprocessing of the data for the model is now simplified. The images in the training dataset were first rescaled using TensorFlow's ImageDataGenerator, with each pixel being normalised to the range [0,1] by dividing 255. The training dataset was also further split into training and validation subsets in an 80:20 ratio, with the validation subset ensuring true generalization of the model before it is evaluated using the training dataset.

All images were then checked to ensure a uniform size of 32×32 pixels, and a batch size of 32 was selected to balance training speed and memory usage.

Since the training dataset of 100,000 images was already sufficiently large, augmentation techniques such as rotation were deliberately left out of preprocessing.

4 Model

To solve this binary classification problem, a simple CNN model will be utilised. This self-built CNN will then be benchmarked against a state-of-the-art pre-trained model, namely the MobileNet v2 model.

Keras, a high-level API of the TensorFlow platform, will be used to build the model.

4.1 Settings

The network begins with a convolutional layer with 8 filters of size 3x3. Following the convolutional layer, a pooling operation of size 2 is applied.

For the updating of weights, the default combination of the Adam optimizer and a 0.001 learning rate is used.

4.2 Evaluation

The model is trained with an epoch count of 3, before being tested on the test dataset with 20,000 images.

During the training phase, the model managed to achieve a decently high accuracy (Table 1).

Epoch	Accuracy	Loss	Validation Accuracy	Validation Loss
1	0.7337	0.5249	0.8204	0.4016
2	0.8352	0.3738	0.8472	0.3563
3	0.8530	0.3458	0.8404	0.3557

Table 1: Results of Training Run

During testing, the model managed to achieve a test accuracy of 0.837 and loss of 0.363.

4.3 Benchmarking

Using a pre-trained CNN model, namely the MobileNet v2, the performance of the built CNN model was tested.

Fine-tuning the MobileNet v2 model using the same training dataset, the following evaluation metrics were obtained:

Epoch	Accuracy	Loss	Validation Accuracy	Validation Loss
1	0.7918	0.4341	0.8881	0.2712
2	0.8898	0.2698	0.9006	0.2418
3	0.9004	0.2442	0.9064	0.2262

Table 2: Results of Training Run for MobileNet v2

As for the test set, an accuracy of 0.9074 and loss of 0.2291 was recorded.

While more desirable metrics were obtained using the pre-trained model, there were still advantages to using a self-built CNN. As a self-built CNN is more lightweight in terms of convolutional layers et cetera, training speeds for he self-built CNN greatly surpassed that of MobileNet v2, which makes it more suitable for low-resource devices.

4.4 GUI

With the main aim of bringing good when building this model, creating a user-friendly interface was also crucial.

Using Gradio, a Python package, a web application with a shareable link was created, allowing users to input images from their local device and outputting the results of whether the image was deemed as real or AI generated, as well as the confidence metric (Images 1,2).

However, a limitation encountered was that running Gradio on Google Colab meant that the link was only accessible for a week before needing to regenerate. To generate a link, only the cells containing the Gradio code and loading of the saved models need to be run, allowing for efficient access.

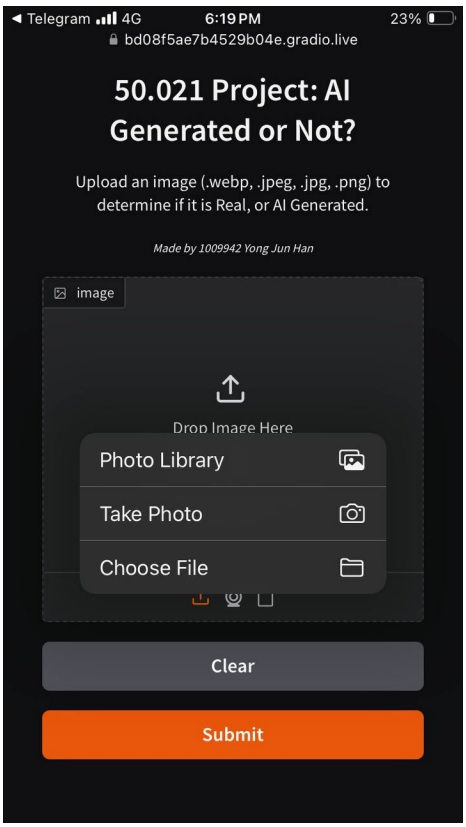

Figure 1: Image 1: GUI

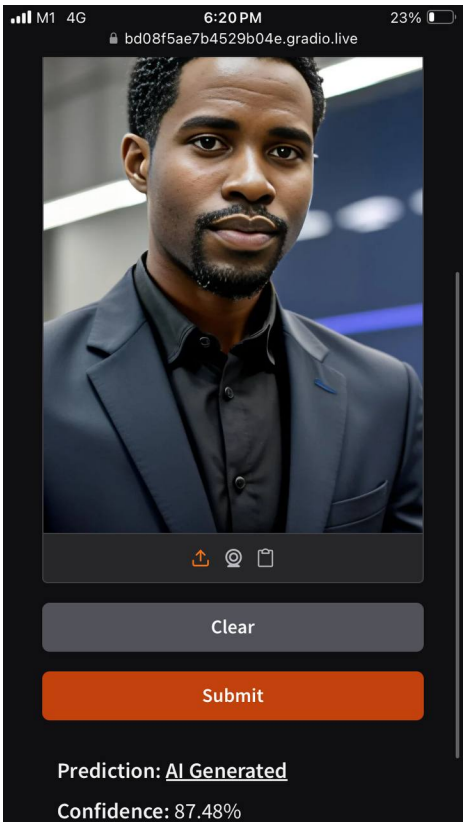

Figure 2: Image 2: Evaluation of selected image